Лабораторная работа №4

Тема: Авторизация и разграничение доступа в Express REST API

Цель: Применить RBAC для Products, Categories и Users; самостоятельно реализовать ограничение доступа к Orders по владельцу.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом express-shop. В проекте должны работать регистрация, login, JWT access token и middleware requireAuth.

Middleware requireAuth после проверки JWT должен создавать req.user как минимум с полями id и role.

> **Главное правило**
>
> Frontend не является границей безопасности. Даже если кнопка скрыта в интерфейсе, backend обязан самостоятельно проверить права пользователя.

1. Проверьте поле role

prisma/schema.prisma

```
model User {
  id           Int        @id @default(autoincrement())
  name         String
  email        String     @unique
  passwordHash String     @map("password_hash")
  role         String     @default("user")
  orders       Order[]

  @@map("users")
}
```

Обычная публичная регистрация должна создавать пользователя с ролью user.

2. Убедитесь, что register не принимает role

services/authService.js

```
export async function registerUser({ name, email, password }) {
  const passwordHash = await hashPassword(password);

  return prisma.user.create({
    data: {
      name,
      email,
      passwordHash
    },
    select: {
      id: true,
      name: true,
      email: true,
      role: true
    }
  });
```

```
}
```

Не доверяйте role из body

Даже если клиент отправит role: admin, публичная регистрация не должна использовать это значение.

3. Создайте обычного пользователя

POST /api/auth/register

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "User One",
  "email": "user1@example.com",
  "password": "Example123!"
}
```

4. Создайте пользователя для роли admin

Зарегистрируйте второй аккаунт обычным способом, затем измените его role непосредственно в доверенной базе данных.

Регистрация будущего admin

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Admin One",
  "email": "admin@example.com",
  "password": "AdminExample123!"
}
```

Назначение admin в PostgreSQL

```
UPDATE users
SET role = 'admin'
WHERE email = 'admin@example.com';
```

Получите новый JWT

Если JWT был получен до изменения role, в старом token останется старое значение role. Выполните login повторно после назначения admin.

5. Реализуйте requireRole

middleware/authMiddleware.js

```
export function requireRole(...allowedRoles) {
  return function (req, res, next) {
    if (!req.user) {
      return res.status(401).json({
        error: "Authentication required"
      });
    }

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        error: "Forbidden"
      });
    }

    next();
  };
}
```

requireRole() должен использоваться после requireAuth, потому что именно requireAuth создаёт доверенный req.user.

6. Защитите Product routes

routes/productRoutes.js

```
router.get("/", getAllProducts);
router.get("/:id", getProductById);

router.post(
  "/",
  requireAuth,
  requireRole("admin"),
  createProduct
);

router.patch(
  "/:id",
  requireAuth,
  requireRole("admin"),
  updateProduct
);

router.delete(
  "/:id",
  requireAuth,
  requireRole("admin"),
  deleteProduct
);
```

Ожидаемая политика Product API

Действие / маршрут	Ожидаемое поведение
GET /api/products	Гость, user, admin — разрешено
POST /api/products	Гость — 401; user — 403; admin — разрешено
PATCH /api/products/:id	Гость — 401; user — 403; admin — разрешено

Действие / маршрут	Ожидаемое поведение
DELETE /api/products/:id	Гость — 401; user — 403; admin — разрешено

7. Проверьте POST /api/products без JWT

```
POST /api/products
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 25000,
  "categoryId": 1,
  "inStock": true
}
```

Ожидаемый HTTP status — 401, потому что Authentication отсутствует.

8. Проверьте POST /api/products с ролью user

```
POST /api/products
Authorization: Bearer <userAccessToken>
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 25000,
  "categoryId": 1,
  "inStock": true
}
```

Ожидаемый HTTP status — 403. Пользователь аутентифицирован, но не имеет административных прав.

9. Проверьте POST /api/products с ролью admin

```
POST /api/products
Authorization: Bearer <adminAccessToken>
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 25000,
  "categoryId": 1,
  "inStock": true
}
```

Запрос должен пройти authorization middleware и создать Product.

10. Разграничьте доступ к Categories

Категории видны всем посетителям магазина. Создание, изменение и удаление категорий разрешены только admin. В этой части работаем с уже существующими обработчиками категорий, если они имеются в проекте.

routes/categoryRoutes.js — добавьте middleware к существующим маршрутам; сохраните принятые в вашем проекте названия контроллеров.

```
import { Router } from "express";
import * as controller from "../controllers/categoryController.js";
import { requireAuth, requireRole } from "../middleware/authMiddleware.js";

const router = Router();

router.get("/", controller.getCategories);
router.post("/", requireAuth, requireRole("admin"), controller.createCategory);

export default router;
```

Если у вас реализованы PATCH /:id и DELETE /:id для категорий, поставьте перед их обработчиками те же requireAuth и requireRole("admin").

Проверьте существующие имена

getCategories и createCategory здесь — пример имён функций. Если в вашем проекте экспортируются другие имена, используйте их. Middleware проверки входных данных, если оно уже есть, оставьте после requireRole.

11. Проверьте Categories как гость, user и admin

Действие / маршрут	Ожидаемое поведение
GET /api/categories без JWT	200, список категорий
POST /api/categories без JWT	401 Authentication required
POST /api/categories с JWT роли user	403 Forbidden
POST /api/categories с JWT роли admin	Успешное создание категории; обычно 201

Для POST используйте JSON с полем name, например {"name":"Мониторы"}. При повторном использовании имени результат зависит от ограничений вашей базы и проверок.

12. Добавьте защищённый маршрут «Мой профиль»

Пользователь должен иметь возможность получить собственные публичные данные. Не передавайте userId в query или body: идентификатор берётся из req.user.id после проверки JWT.

routes/userRoutes.js

```
import { Router } from "express";
import * as controller from "../controllers/userController.js";
import { requireAuth, requireRole } from "../middleware/authMiddleware.js";

const router = Router();
router.get("/me", requireAuth, controller.getMe);
router.get("/", requireAuth, requireRole("admin"), controller.getUsers);

export default router;
```

controllers/userController.js

```
import { prisma } from "../config/prisma.js";

const safeUser = { id: true, name: true, email: true, role: true };

export async function getMe(req, res, next) {
  try {
    const user = await prisma.user.findUnique({
      where: { id: req.user.id },
      select: safeUser
    });
    if (!user) return res.status(404).json({ error: "User not found" });
    res.json(user);
  } catch (error) {
    next(error);
  }
}

export async function getUsers(req, res, next) {
  try {
    const users = await prisma.user.findMany({ select: safeUser });
    res.json(users);
  } catch (error) {
    next(error);
  }
}
```

Подключите маршруты в app.js (или в файле, где у вас подключаются Express routers):

```
import userRoutes from "./routes/userRoutes.js";
app.use("/api/users", userRoutes);
```

Не передавайте passwordHash

Prisma Client может вернуть поле passwordHash при запросе без select. Поэтому используйте select: { id, name, email, role } и для /me, и для списка пользователей.

13. Проверьте API пользователей

Действие / маршрут	Ожидаемое поведение
GET /api/users/me без JWT	401
GET /api/users/me с JWT user	200; данные только текущего пользователя
GET /api/users с JWT user	403
GET /api/users с JWT admin	200; безопасный список пользователей без passwordHash

Для учебного проекта GET /api/users предназначен только для администратора. При проверке убедитесь, что в каждом JSON-ответе отсутствуют password и passwordHash.

14. Сведите правила доступа в единую таблицу

Действие / маршрут	Ожидаемое поведение
GET /api/products и GET /api/categories	Публичный доступ
POST/PATCH/DELETE /api/products	Только admin
POST/PATCH/DELETE /api/categories (если маршруты реализованы)	Только admin
GET /api/users/me	Любой аутентифицированный пользователь — только себя
GET /api/users	Только admin
POST /api/orders; GET /api/orders/my; GET /api/orders/my/:id	Самостоятельная реализация, только с JWT и проверкой владельца

15. Самостоятельная работа: реализуйте доступ к Orders

С этого пункта приводится алгоритм и ориентиры, а не готовые реализации Order Controller и Order Service. Работа с заказами опирается на вашу схему Prisma; если Order и OrderItem ещё не созданы, сначала спроектируйте их как часть курсовой работы.

15.1. Подготовьте модели заказов

- Order: id, ссылка на пользователя (userId), статус заказа; при необходимости дата создания.
- OrderItem: ссылка на заказ, ссылка на товар (productId), количество (quantity); при необходимости цена на момент покупки.
- Связи: User → Order (один ко многим); Order → OrderItem (один ко многим); Product → OrderItem (один ко многим).
- Согласуйте имена полей и моделей с вашей реальной schema.prisma; в старых проектах модели могут называться orders и order_items.

> **Важно для Prisma**
>
> Код prisma.order доступен, только если в schema.prisma действительно есть model Order. Если модель названа orders, клиент использует prisma.orders. Не запускайте migrate reset ради этого задания: он удаляет данные.

15.2. Создайте маршруты заказов

routes/orderRoutes.js — составьте самостоятельно. Маршруты, которые требуется реализовать:

Действие / маршрут	Ожидаемое поведение
POST /api/orders	Только с JWT; создать заказ для req.user.id
GET /api/orders/my	Только с JWT; получить свои заказы
GET /api/orders/my/:id	Только с JWT; получить один свой заказ
GET /api/orders	Дополнительно: admin может видеть все заказы

Подсказка по структуре маршрута (только один пример):

```
router.get("/my", requireAuth, controller.getMyOrders);
// Самостоятельно добавьте POST / и GET /my/:id
// Важно: /my должен быть объявлен до /:id, если такой маршрут есть.
```

15.3. Создайте заказ для текущего пользователя

- Получите массив позиций заказа из req.body.items.
- Проверьте, что массив не пустой, а productId и quantity корректны; проверьте существование товаров.
- Возьмите идентификатор владельца только из req.user.id, НЕ из req.body.userId.
- Создайте Order и связанные OrderItem через Prisma; при нескольких зависимых записях используйте транзакцию или nested write.
- Верните созданный заказ без секретных полей пользователя.

Подсказка: ключевое выражение в контроллере — req.user.id, например вызов service:

```
const order = await createOrder(req.user.id, req.body.items);
// createOrder реализуйте самостоятельно
```

Тестовый запрос:

```
POST /api/orders
Authorization: Bearer <userAccessToken>
Content-Type: application/json

{
  "items": [ { "productId": 1, "quantity": 2 } ]
}
```

15.4. Реализуйте получение только своих заказов

Подсказка для Prisma: в условии поиска должен присутствовать userId: req.user.id. Для получения одного заказа нужно одновременно ограничить поиск по id и userId.

```
// Ориентир для WHERE, не полный код:
where: { userId: req.user.id }

// Один конкретный заказ:
where: { id: Number(req.params.id), userId: req.user.id }
```

Если поиск своего заказа не вернул запись — отвечайте 404. Это позволяет одинаково обрабатывать чужой и несуществующий заказ.

15.5. Проверьте невозможность подмены владельца

- Зарегистрируйте пользователя A и пользователя B и войдите под каждым, получив разные JWT.
- Под токеном A создайте заказ и запишите его id.
- Под токеном B выполните GET /api/orders/my/:id с id заказа A — ожидается 404.
- Под токеном A запросите этот же заказ — ожидается 200.

- Под токеном A отправьте POST /api/orders с дополнительным userId пользователя B; сервер должен игнорировать это поле и связать заказ с пользователем A.
- Проверьте содержимое базы SQL-запросом или pgAdmin, убедившись, что владельцем остался A.

16. Дополнительное самостоятельное задание

- Для products реализуйте (или проверьте) PATCH /:id и DELETE /:id только для admin.
- Для categories аналогично защитите изменение и удаление, если эти маршруты существуют.
- Вместо списка всех пользователей разрешите GET /api/users только роли admin.
- Объясните на своём API различие между ответами 401, 403 и 404.

Требования к результату

- Пункты 1–9 выполнены без изменения логики примеров.
- Чтение products/categories публичное; запись и изменение каталога требуют роли admin.
- GET /api/users/me показывает только текущего пользователя; полный список доступен только admin.
- Ни один пользовательский HTTP-ответ не содержит passwordHash.
- Order использует владельца из req.user.id; пользователь получает только свои заказы.
- Показана проверка попытки получить чужой заказ и подменить userId.

Критерии оценивания

Действие / маршрут	Ожидаемое поведение
1. Аутентификация и роли	requireAuth и requireRole выполняют проверки в правильном порядке
2. Products	user не может изменять товары, admin может
3. Categories	Гость может читать, admin может создавать
4. Users	/me доступен владельцу, список — admin, без passwordHash
5. Orders	Самостоятельно реализовано создание и получение своих заказов
6. Проверка защиты	Чужой заказ недоступен; userId из тела запроса игнорируется
7. Объяснение	Студент различает 401, 403 и 404